

CS536 — Spring 2026 — TCP Expo

Group: Youngjun Yoo, Dylan Boyer, Alejandro Griffith, Douglas Muir, Amit Manchella

March 10, 2026

1 TCP Expo Overview

Relevant Links

[HW 2 Repo Link](#)

[HW 3 Repo Link](#)

****INSTRUCTIONS ON HOW TO SET UP OUR KERNEL MODULE****

Strategy

TCP Expo is a delay-based congestion control algorithm we built as a kernel module for CS536. Instead of waiting for packet loss to know the network is congested (like Reno and CUBIC do), Expo watches the round-trip time. When RTT starts climbing, it means packets are sitting in router queues — the network is getting full even though nothing has been dropped yet.

The core idea is momentum: we track how much RTT has inflated compared to the baseline (the lowest RTT we've seen). If RTT is still close to baseline, we grow the window aggressively. If it's spiking up, we slow down. If it spikes past 75% inflation, we do a small reduction called a Negative Pulse (thanks to the professor advice) — dropping the window by about 3% to reduce the queue back down without throwing away throughput.

There are three zones:

- Clean zone ($< 30\%$ RTT inflation) — grow fast, the pipe has room
- Build zone (30–75%) — some queuing, grow carefully
- Expo zone ($> 75\%$) — too much delay, fire a Negative Pulse

This lets Expo fill the pipe without flooding it. Furthermore, in the clean zone we have an acceleration system: the longer the connection goes without hitting congestion, the faster we let the window grow (up to 16x the base rate). We also wanted to have fun with it and experimented a bit—we start with an initial window of 20 segments instead of the default 10, which gets data moving faster on high-bandwidth links.

If you want more information on how our algorithm works, [click here](#).

2 Testing with suite from HW 2

We used the testing suite from HW2 to evaluate TCP Expo against three congestion control algorithms: BBR, CUBIC, and Reno across two test durations (10 seconds and 60 seconds). All tests were conducted over the same endpoints to ensure we had consistent results. The full repository of all data is available at these links:

- [10s test data](#)
- [60s test data](#)

BBR 10 second run

Table 1: Network metrics

Host	Country	Tput (Mbps)	RTT (ms)	Lost	Retrans
speedtest.nyc1.us.leaseweb.net	US	778.08	27.85	0	3
37.19.206.20	US	774.59	27.69	0	8
107.1.167.50	US	352.00	59.70	57	76
ping.online.net	FR	187.36	109.25	0	0
sgp.proof.ovh.net	SG	85.76	218.52	0	2

Cubic 10 second run

Table 2: Network metrics

Host	Country	Tput (Mbps)	RTT (ms)	Lost	Retrans
speedtest.nyc1.us.leaseweb.net	US	777.02	27.88	0	10
37.19.206.20	US	703.99	29.59	1	113
107.1.167.50	US	355.50	59.53	0	0
ping.online.net	FR	204.97	100.08	0	2
sgp.proof.ovh.net	SG	76.66	242.76	0	1

Expo 10 second run

Table 3: Network metrics

Host	Country	Tput (Mbps)	RTT (ms)	Lost	Retrans
37.19.206.20	US	800.65	27.01	0	2
speedtest.nyc1.us.leaseweb.net	US	777.88	27.89	0	2
107.1.167.50	US	355.30	59.69	0	0
ping.online.net	FR	197.00	100.30	482	769
sgp.proof.ovh.net	SG	73.99	233.60	0	21

Reno 10 second run

Table 4: Network metrics

Host	Country	Tput (Mbps)	RTT (ms)	Lost	Retrans
37.19.206.20	US	778.37	27.78	0	3
speedtest.nyc1.us.leaseweb.net	US	315.99	1656.40	0	5
107.1.167.50	US	267.35	78.23	0	2
ping.online.net	FR	196.08	104.63	0	0
sgp.proof.ovh.net	SG	77.57	239.57	0	0

BBR 60 second run

Table 5: Network metrics

Host	Country	Tput (Mbps)	RTT (ms)	Lost	Retrans
speedtest.shinternet.ch	CH	184.72	117.08	126	314

Cubic 60 second run

Table 6: Network Metrics

Host	Country	Tput (Mbps)	RTT (ms)	Lost	Retrans
speedtest.shinternet.ch	CH	179.75	120.95	2	12

Expo 60 second run

Table 7: Network metrics

Host	Country	Tput (Mbps)	RTT (ms)	Lost	Retrans
speedtest.shinternet.ch	CH	185.66	117.45	0	0

Reno 60 second run

Table 8: Network metrics

Host	Country	Tput (Mbps)	RTT (ms)	Lost	Retrans
speedtest.shinternet.ch	CH	179.61	121.05	40	102

Analysis

From the 10 second results, Expo outperforms the other three algorithms when it comes to throughput while performing similarly for round-trip times. Despite this, Expo loses and retransmits an exorbitant amount of packets in some cases during the 10 second test. This inconsistency is likely due to Expo’s aggressive slow-start ramping resulting in the buffer becoming overloaded.

In the 60 second tests, however, Expo has a much stronger performance. Expo showcased a higher throughput than all other algorithms as well as a faster RTT for all algorithms except for BBR. Expo also lost and retransmitted no packets, demonstrating a much stabler connection than the other algorithms which underwent substantial packet loss/retransmissions. This is likely due to Expo’s proactive CWND size reduction logic—we don’t wait for packet loss before making adjustments!

Overall, despite its flaws, Expo’s throughput beat all others when using the HW2 testing suite. We were super stoked about our results! Naturally, we wanted to test our algorithm further, as we were especially curious about how our algorithm dealt with multiple streams and fairness. So, the ensuing parts of this paper are devoted to additional tests and data to complement our existing tests.

3 Extended Testing and Analysis

We used **Flent** (**F**LExible **N**etwork **T**ester) to rigorously evaluate TCP Expo against three congestion control algorithms: BBR, CUBIC, and Reno across two test durations (30 seconds and 60 seconds) in both single-stream and 8-stream configurations. All tests were conducted over the same bottleneck link to make sure we had consistent results. The full repository of all plots and raw CSV data is available at these links:

- [30s test data \(plots + CSV\)](#)
- [60s test data \(plots + CSV\)](#)

3.1 Single-Stream Results

30-Second Single-Stream Throughput

Table 9: Single-stream comparison — 30-second tests

Algorithm	Avg TP (Mbps)	Med TP (Mbps)	P99 TP (Mbps)	Avg RTT (ms)	Med RTT (ms)	RTT Var (ms)
BBR	92.40	101.55	237.97	48.47	45.69	1.80
CUBIC	91.28	90.40	190.18	40.80	41.71	1.24
Expo	92.43	94.13	159.92	50.43	53.37	2.15
Reno	90.25	86.33	200.26	42.53	40.29	1.55

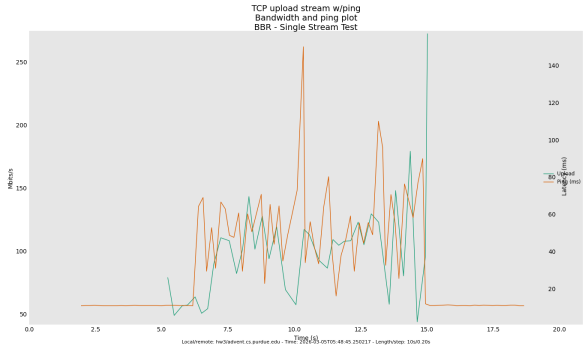
At 30 seconds with a single flow, all four algorithms achieve a resonable throughput in the 90–92 Mbps range. Expo gets the highest out of the field with the highest average throughput (92.43 Mbps), essentially tied with BBR (92.40 Mbps). However, Expo’s average RTT (50.43 ms) is higher than CUBIC’s (40.80 ms) and Reno’s (42.53 ms), indicating queuing caused by its aggressive window growth in the clean zone. A tradeoff intentionally made.

60-Second Single-Stream Throughput

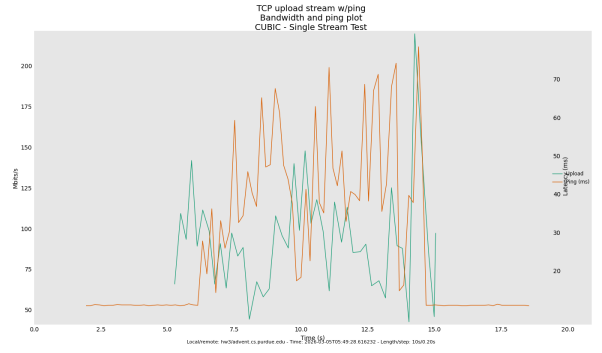
Table 10: Single-stream comparison — 60-second tests

Algorithm	Avg TP (Mbps)	Med TP (Mbps)	P99 TP (Mbps)	Avg RTT (ms)	Med RTT (ms)	RTT Var (ms)
BBR	84.46	84.14	179.56	45.33	45.25	1.66
CUBIC	100.24	92.61	222.75	38.55	36.26	1.30
Expo	96.41	90.63	252.13	39.56	37.10	1.55
Reno	97.02	89.16	224.04	37.98	37.41	1.40

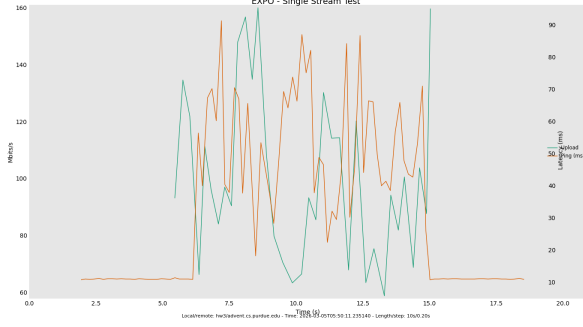
With more time, we can see that the results change. CUBIC achieves the highest average of 100.24 Mbps, followed by Reno (97.02 Mbps) and Expo (96.41 Mbps). Expo achieves the **highest P99 throughput** of 252.13 Mbps, which is due to its aggressive acceleration in the clean zone, allowing stronger performance. Expo’s RTT also drops relative to its 30-second run (39.56 ms vs. 50.43 ms), indicating that its *rtt_min aging* mechanism stabilizes over longer flows.



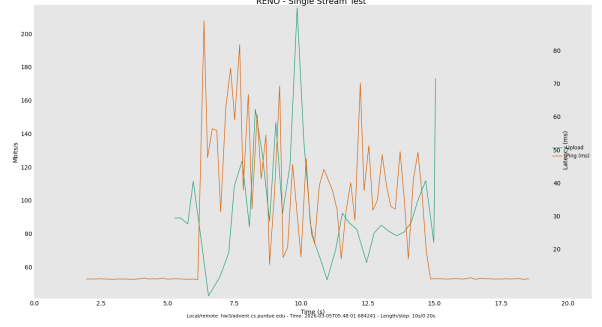
(a) BBR



(b) CUBIC

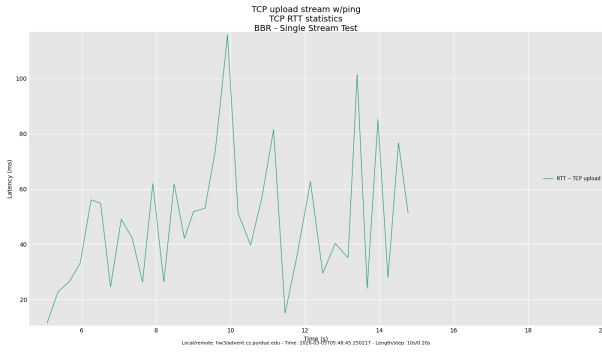


(c) Expo

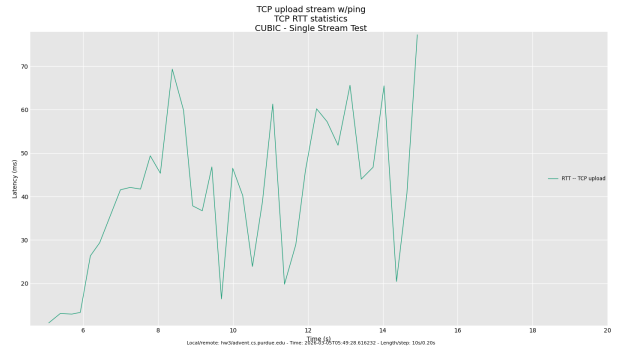


(d) Reno

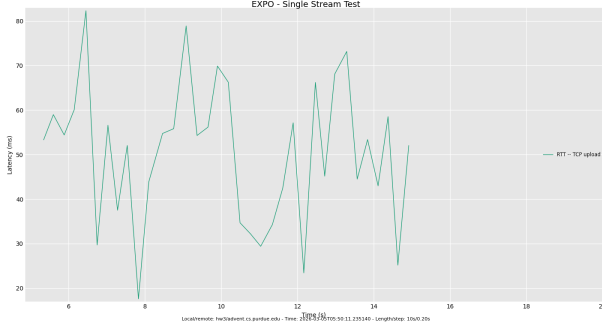
Figure 1: Single-stream throughput over 30 seconds for all four algorithms.



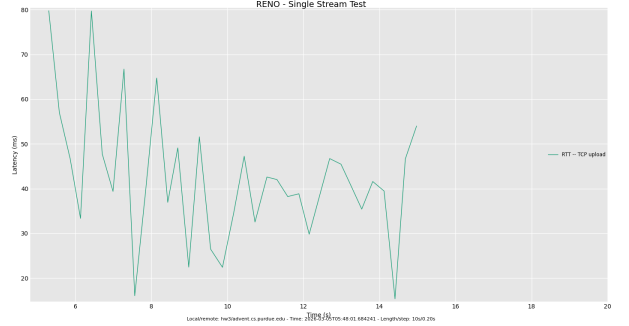
(a) BBR



(b) CUBIC

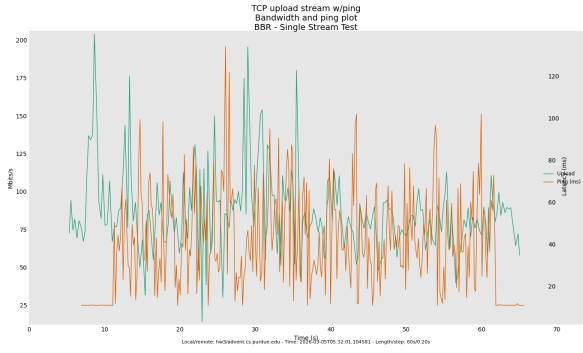


(c) Expo

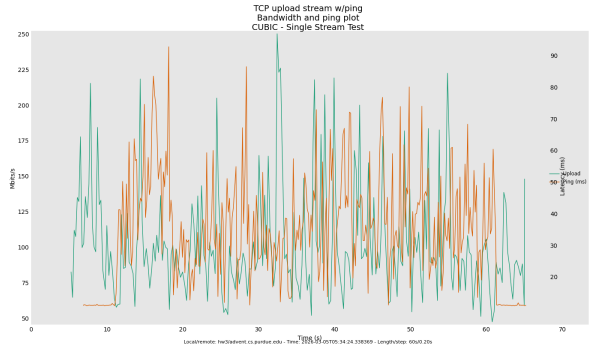


(d) Reno

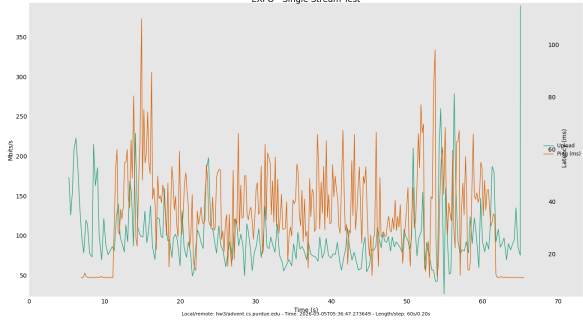
Figure 2: Single-stream RTT over 30 seconds for all four algorithms.



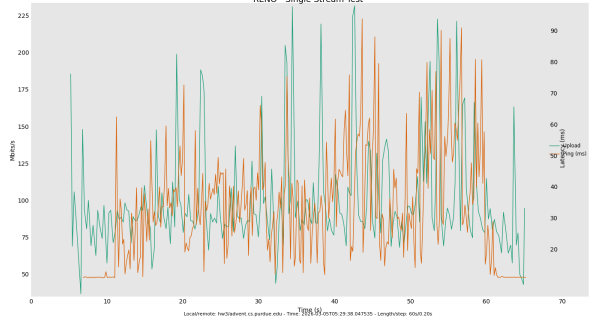
(a) BBR



(b) CUBIC

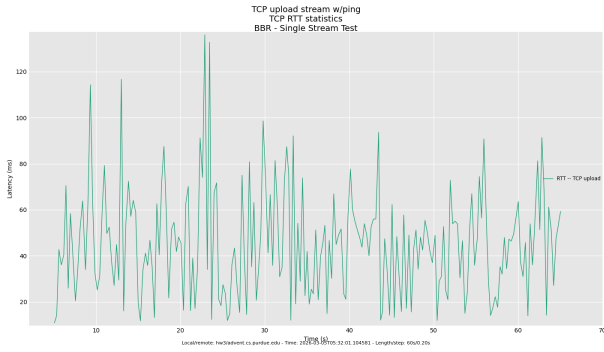


(c) Expo

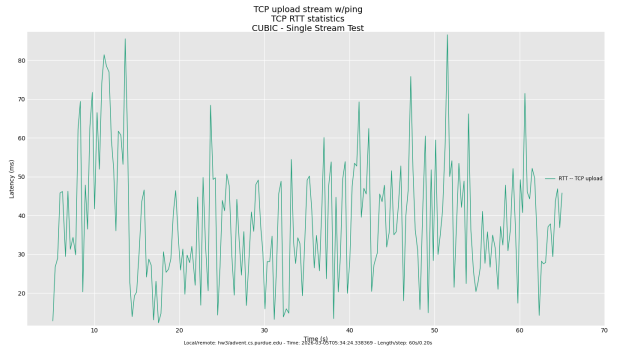


(d) Reno

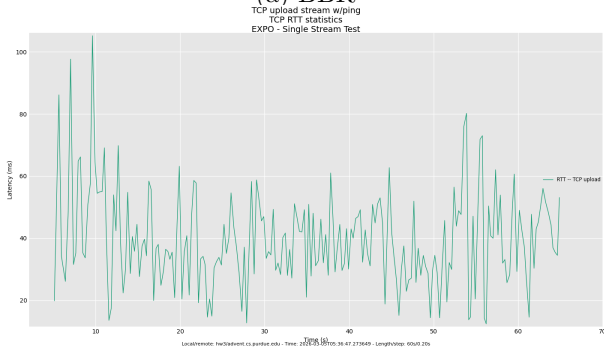
Figure 3: Single-stream throughput over 60 seconds for all four algorithms.



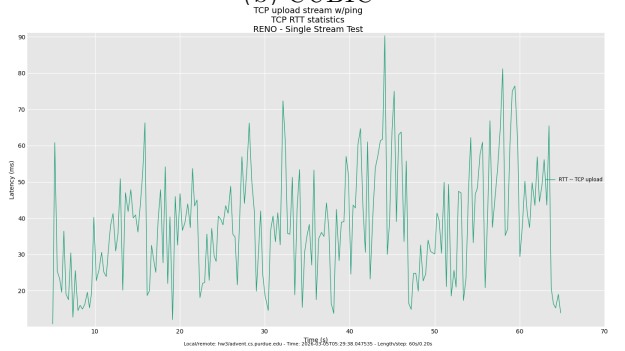
(a) BBR



(b) CUBIC



(c) Expo



(d) Reno

Figure 4: Single-stream RTT over 60 seconds for all four algorithms.

3.2 Multi-Stream (8 Flows) Results

In Multi-stream, we focused on testing fairness, as we believe it is the most important.

30-Second Multi-Stream

Table 11: 8-stream comparison — 30-second tests

Algorithm	Total TP (Mbps)	Per-Flow Avg (Mbps)	Per-Flow Min (Mbps)	Per-Flow Max (Mbps)	Jain's Fairness
BBR	77.32	9.66	8.46	10.97	0.9935
CUBIC	97.80	12.22	6.19	24.61	0.8498
Expo	83.41	10.43	6.81	13.42	0.9682
Reno	98.71	12.34	10.32	15.56	0.9810

At 30 seconds, we can see that Reno leads in total throughput (98.71 Mbps) followed closely by CUBIC (97.80 Mbps). Expo lowers to 83.41 Mbp since the acceleration, and *rtt_min* aging mechanisms haven't fully started. However, the fairness picture is revealing: **CUBIC's Jain's fairness drops to 0.8498**, with one flow grabbing 24.61 Mbps while another gets only 6.19 Mbps; a 4× disparity. Expo achieves much better fairness at 0.9682, and BBR leads in fairness at 0.9935.

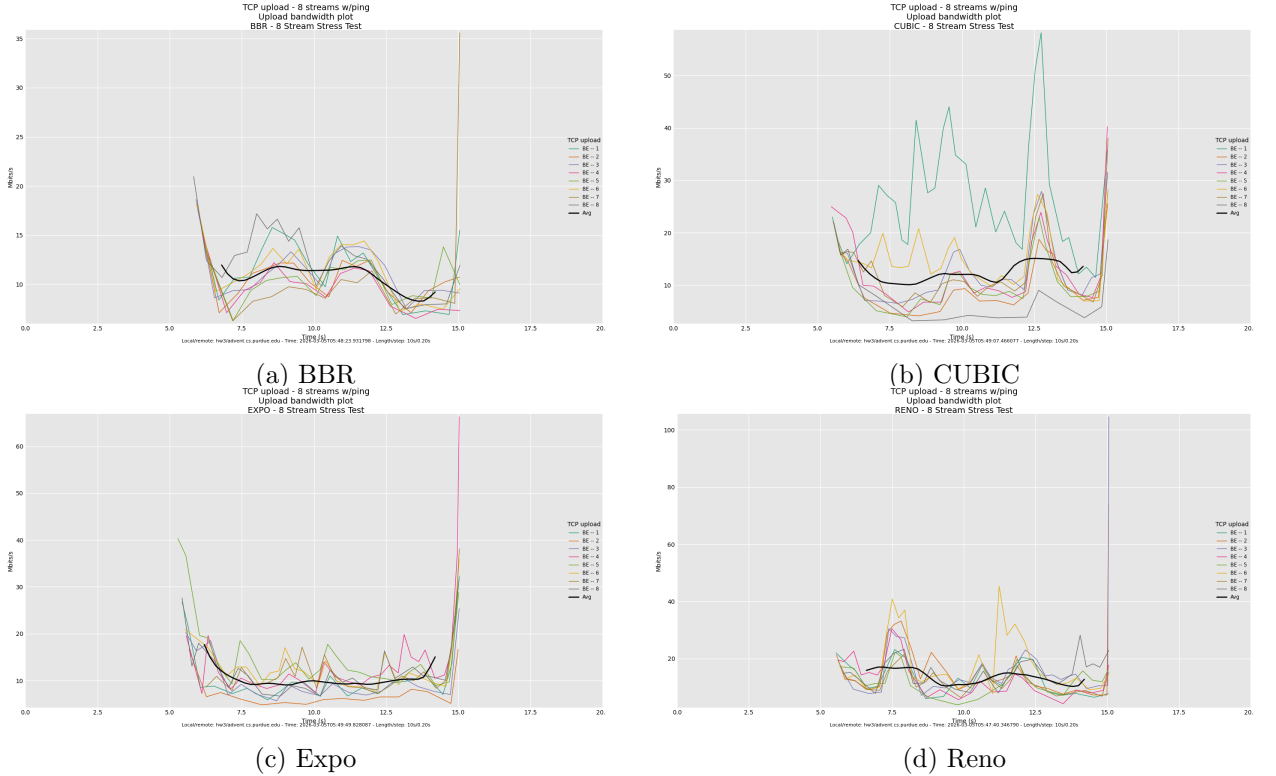


Figure 5: 8-stream throughput over 30 seconds for all four algorithms.

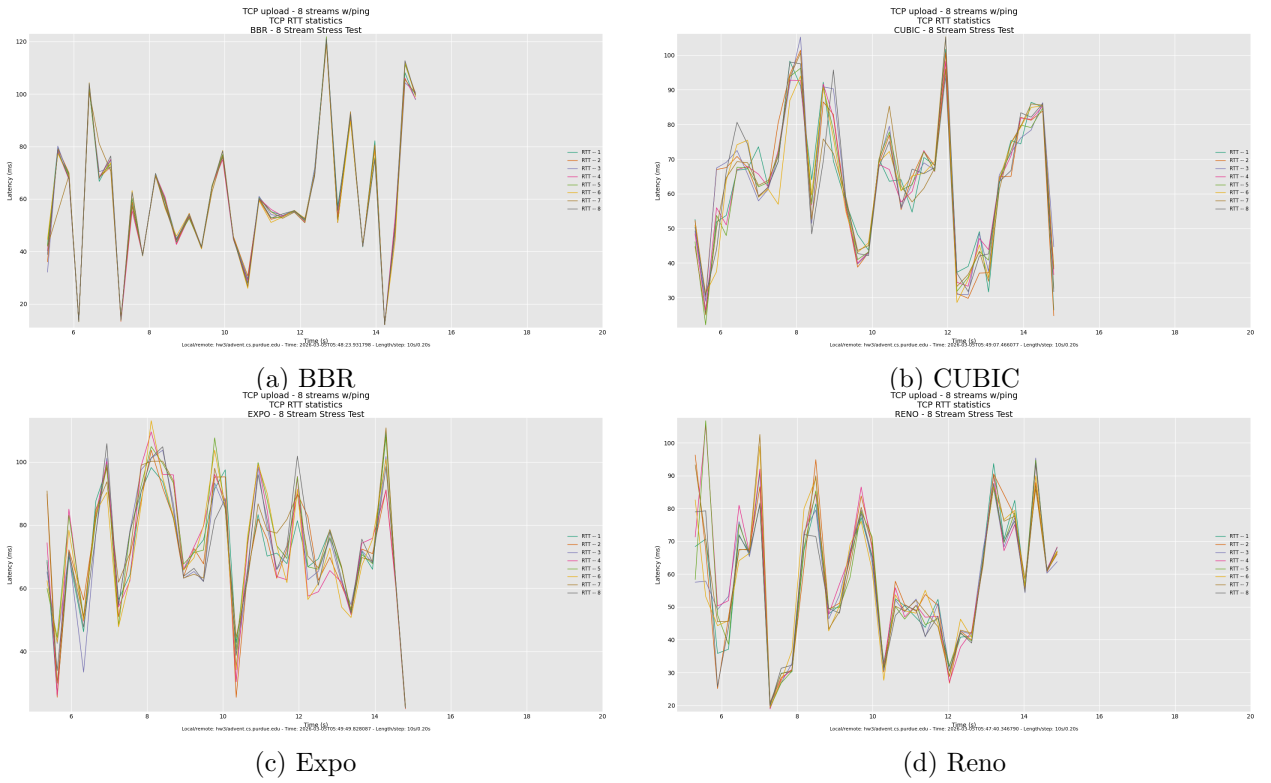


Figure 6: 8-stream RTT over 30 seconds for all four algorithms.

30-Second Per-Flow Breakdown

Table 12: Per-flow throughput (Mbps) — 30-second 8-stream tests

Algorithm	F1	F2	F3	F4	F5	F6	F7	F8
BBR	9.84	9.62	10.43	8.79	9.19	10.02	8.46	10.97
CUBIC	24.61	9.01	11.84	11.40	9.58	13.83	11.34	6.19
Expo	9.45	6.81	9.09	11.53	13.42	11.80	10.13	11.18
Reno	11.48	13.16	13.11	10.96	10.32	15.56	10.46	13.66

The per-flow breakdown at 30 seconds highlights CUBIC’s fairness problem: Flow 1 receives 24.61 Mbps, more than double most other flows, while Flow 8 is starved at 6.19 Mbps. Expo’s flows range from 6.81 to 13.42 Mbps, which is a much closer spread. BBR has the closest spread (8.46–10.97 Mbps) but at the cost of lower total throughput.

60-Second Multi-Stream

Table 13: 8-stream comparison — 60-second tests

Algorithm	Total TP (Mbps)	Per-Flow Avg (Mbps)	Per-Flow Min (Mbps)	Per-Flow Max (Mbps)	Jain's Fairness
BBR	89.79	11.22	9.92	13.89	0.9884
CUBIC	114.64	14.33	12.59	16.42	0.9904
Expo	112.52	14.06	13.68	14.43	0.9998
Reno	97.19	12.15	10.86	13.71	0.9931

Now, the cool part, the 60-second test, is where TCP Expo truly shows that it may actually work. CUBIC achieves the highest total throughput at 114.64 Mbps, but Expo is within 2% at 112.52 Mbps, and **Expo achieves a Jain's fairness index of 0.9998**, which is nearly 1!!!. Expo's per-flow range is just 13.68–14.43 Mbps, a difference of only 0.75 Mbps across all 8 streams. By comparison:

- CUBIC's spread is 3.83 Mbps (12.59–16.42), fairness 0.9904
- Reno's spread is 2.85 Mbps (10.86–13.71), fairness 0.9931
- BBR's spread is 3.97 Mbps (9.92–13.89), fairness 0.9884

This shows that Expo's delay-based approach balances competing flows: since all flows see roughly the same RTT inflation on a shared bottleneck, they all converge to the same sending rate.

60-Second Per-Flow Breakdown

Table 14: Per-flow throughput (Mbps) — 60-second 8-stream tests

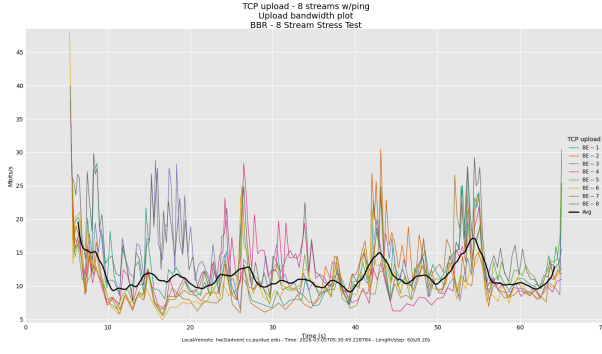
Algorithm	F1	F2	F3	F4	F5	F6	F7	F8
BBR	11.66	10.33	12.03	11.02	10.75	9.92	10.19	13.89
CUBIC	13.03	14.20	12.59	15.01	16.41	12.95	16.42	14.03
Expo	14.43	13.68	13.98	14.17	14.04	14.02	14.12	14.08
Reno	10.94	10.86	13.71	11.89	13.35	11.30	12.67	12.47

As mentioned above, Expo's 60-second per-flow data is really uniform.

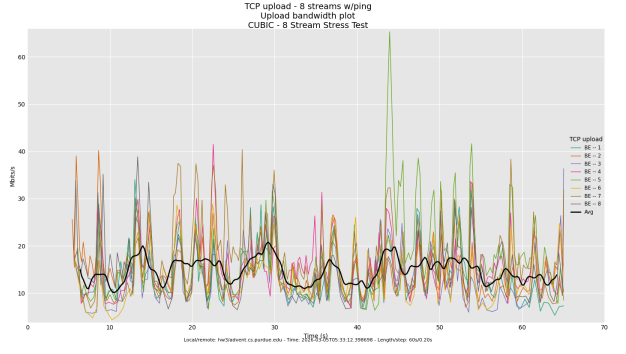
3.3 Congestion Window and Diagnosis Plots

The congestion window (CWND) plots reveal how each algorithm manages its sending window. The diagnosis plots (available for 8-stream tests) provide additional insight into internal algorithm behavior.

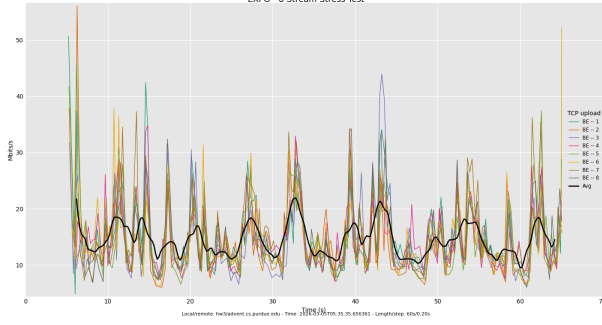
Expo's CWND traces we believe (Figure 9c) are slightly smoother and more tightly grouped than CUBIC's (Figure 9b), which shows more per-flow variance. This is the result of Expo's continuous delay-based adjustment.



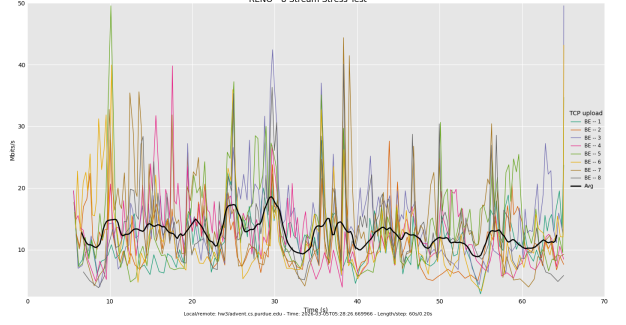
(a) BBR



(b) CUBIC

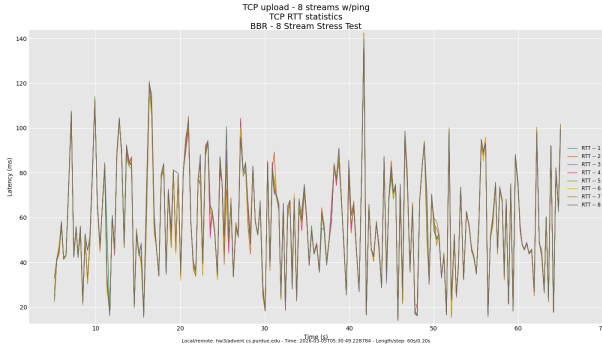


(c) Expo

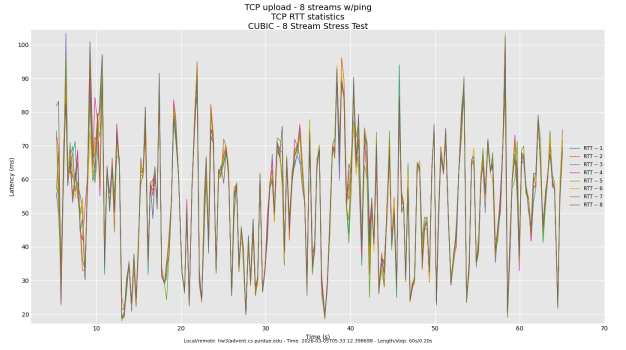


(d) Reno

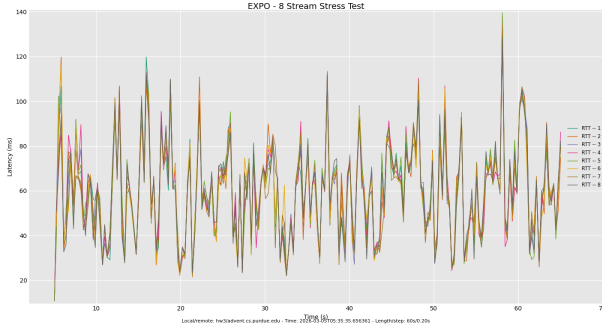
Figure 7: 8-stream throughput over 60 seconds for all four algorithms.



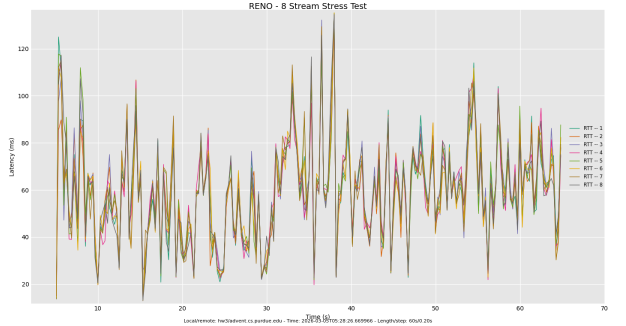
(a) BBR



(b) CUBIC

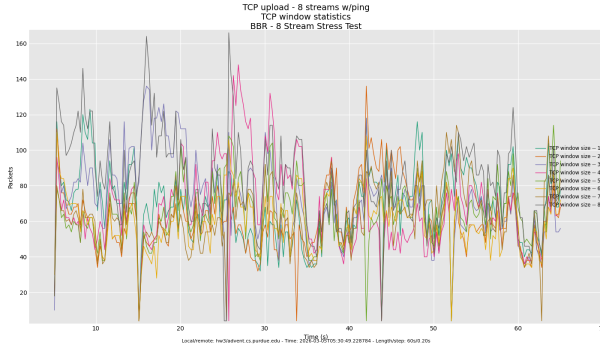


(c) Expo

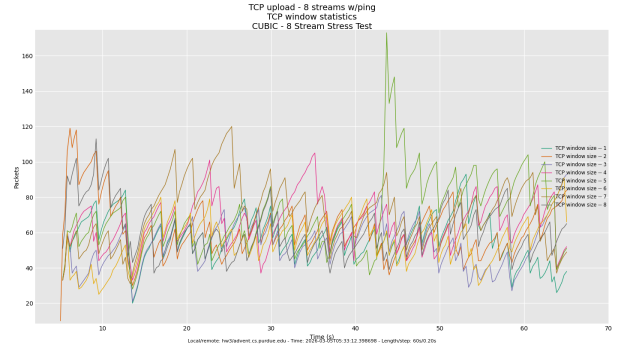


(d) Reno

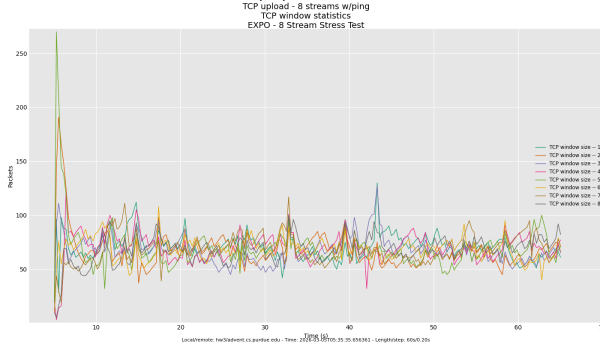
Figure 8: 8-stream RTT over 60 seconds for all four algorithms.



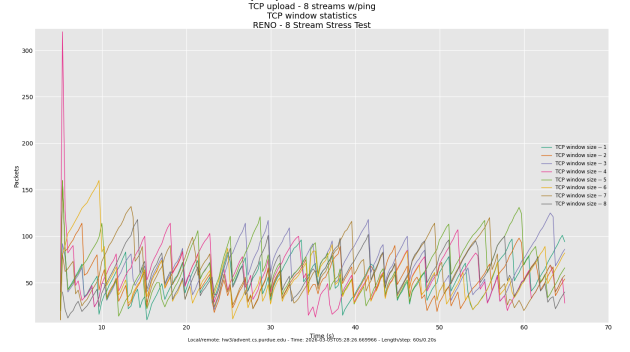
(a) BBR



(b) CUBIC

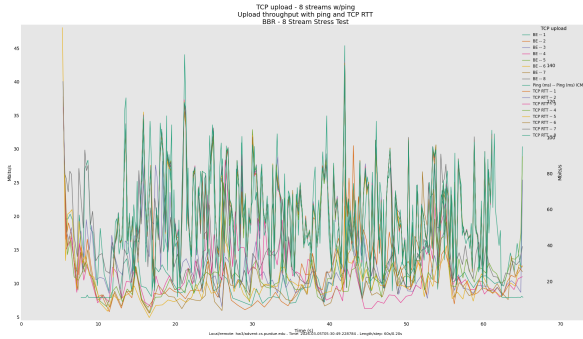


(c) Expo

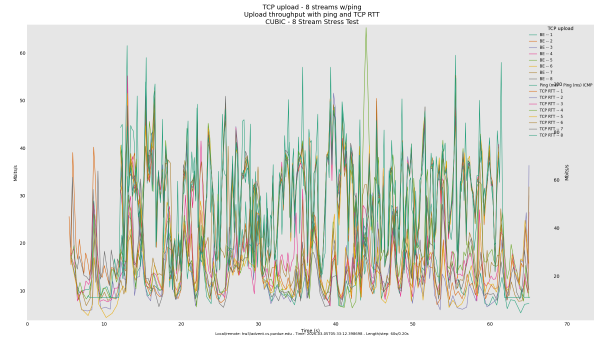


(d) Reno

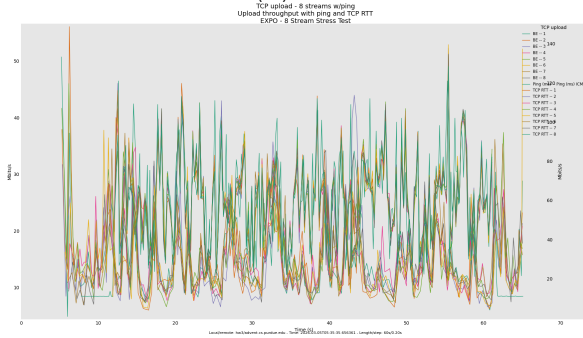
Figure 9: 8-stream CWND over the 60-second test for all four algorithms.



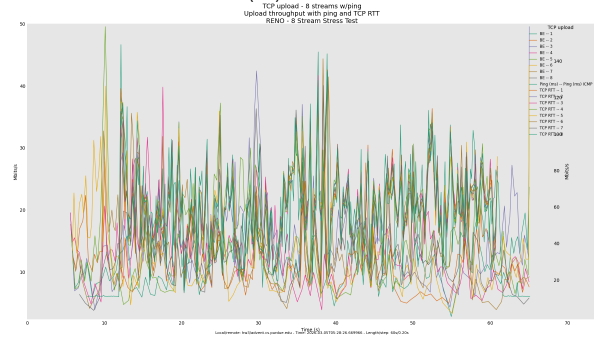
(a) BBR



(b) CUBIC



(c) Expo



(d) Reno

Figure 10: 8-stream diagnostic plots over 60 seconds for all four algorithms.

3.4 Early Iteration: AWS Testing

To showcase how we tried multiple approaches before landing on our final momentum and delay-based algorithm, here is an early iteration that performed poorly. Before the professor set up a Purdue endpoint, we used an AWS instance to test how our algorithm handled multiple competing flows on a single bottleneck link:

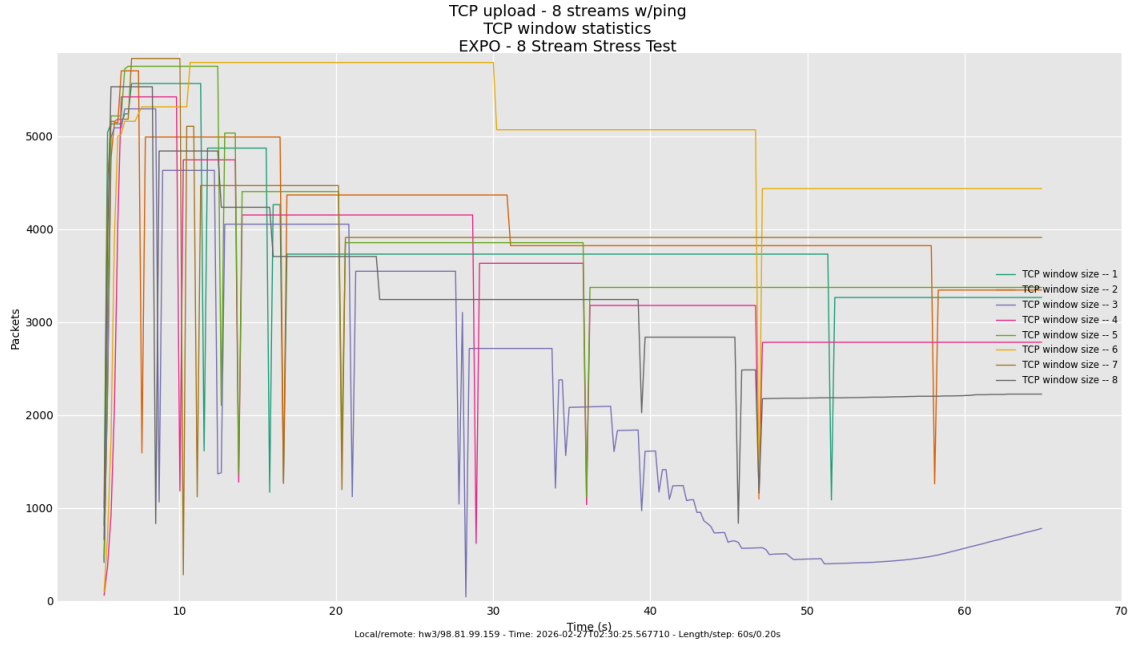


Figure 11: Early prototype testing on AWS

After we saw one of the clients was completely starved of bandwidth, it was clear that our initial approach had a serious fairness issue. We completely rewrote the algorithm after this, which led to the momentum-based design described in Section 1 and the near-perfect fairness results shown above.